

Lab Assignment 11.1

Name : Aditya Raj(46)

Roll No: 2303A53031

Task Description #1 – Stack Implementation

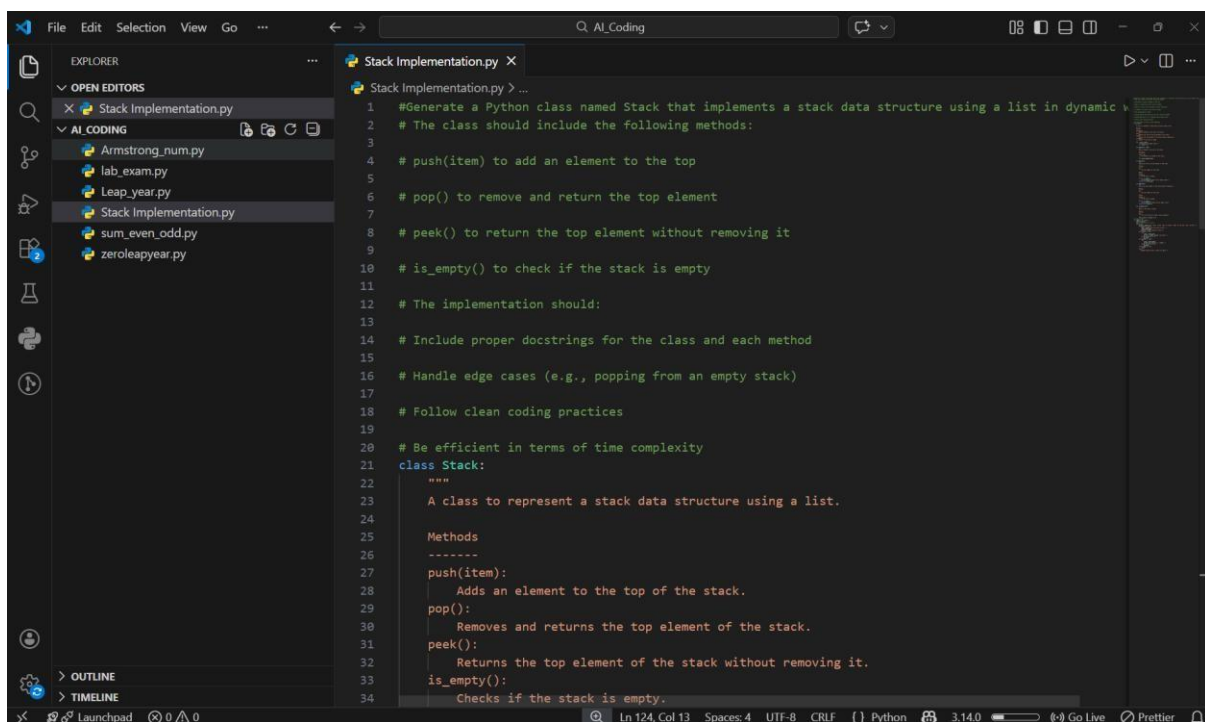
Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.

Sample Input Code: class

Stack: pass

Expected Output: • A functional stack implementation with all required methods and docstrings.

Code:

A screenshot of a code editor window titled 'Stack Implementation.py'. The editor shows a Python class 'Stack' implemented using a list. The class includes methods 'push(item)', 'pop()', 'peek()', and 'is_empty()'. The code is well-commented with docstrings for each method and a class-level docstring. The left sidebar shows the file explorer with a folder 'AI_CODING' containing several files, including 'Stack Implementation.py'. The bottom status bar indicates the file is at line 124, column 13, using UTF-8 encoding and CRLF line endings. The status bar also shows the Python version as 3.14.0 and the Prettier formatter is active.

```
1 #Generate a Python class named Stack that implements a stack data structure using a list in dynamic
2 # The class should include the following methods:
3
4 # push(item) to add an element to the top
5
6 # pop() to remove and return the top element
7
8 # peek() to return the top element without removing it
9
10 # is_empty() to check if the stack is empty
11
12 # The implementation should:
13
14 # Include proper docstrings for the class and each method
15
16 # Handle edge cases (e.g., popping from an empty stack)
17
18 # Follow clean coding practices
19
20 # Be efficient in terms of time complexity
21 class Stack:
22     """
23     A class to represent a stack data structure using a list.
24
25     Methods
26     -----
27     push(item):
28         Adds an element to the top of the stack.
29     pop():
30         Removes and returns the top element of the stack.
31     peek():
32         Returns the top element of the stack without removing it.
33     is_empty():
34         Checks if the stack is empty.
```

```
Stack Implementation.py X
Stack Implementation.py > ...
21 class Stack:
34     """ Checks if the stack is empty.
35     """
36     def __init__(self):
37         """Initialize an empty stack."""
38         self.stack = []
39
40     def push(self, item):
41         """
42         Add an element to the top of the stack.
43
44         Parameters
45         -----
46         item : any
47             The element to be added to the stack.
48         """
49         self.stack.append(item)
50
51     def pop(self):
52         """
53         Remove and return the top element of the stack.
54
55         Returns
56         -----
57         any
58             The top element of the stack.
59
60         Raises
61         -----
62         IndexError
63             If the stack is empty.
64         """
65         if self.is_empty():
66             raise IndexError("Pop from an empty stack")
```

```
Stack Implementation.py > ...
21 class Stack:
51     def pop(self):
65         if self.is_empty():
66             raise IndexError("Pop from an empty stack")
67         return self.stack.pop()
68
69     def peek(self):
70         """
71         Return the top element of the stack without removing it.
72
73         Returns
74         -----
75         any
76             The top element of the stack.
77
78         Raises
79         -----
80         IndexError
81             If the stack is empty.
82         """
83         if self.is_empty():
84             raise IndexError("Peek from an empty stack")
85         return self.stack[-1]
86
87     def is_empty(self):
88         """
89         Check if the stack is empty.
90
91         Returns
92         -----
93         bool
94             True if the stack is empty, False otherwise.
95         """
96         return len(self.stack) == 0
```

```
Stack Implementation.py X
Stack Implementation.py > ...
21 class Stack:
87     def is_empty(self):
96         return len(self.stack) == 0
97 # Example usage:
98 if __name__ == "__main__":
99     stack = Stack()
100     #give dynamic input to stack
101     while True:
102         action = input("Enter 'push' to add, 'pop' to remove, 'peek' to view top, 'exit' to quit: ")
103         if action == 'push':
104             item = input("Enter the item to push: ")
105             stack.push(item)
106             print(f"'{item}' pushed to stack.")
107         elif action == 'pop':
108             try:
109                 item = stack.pop()
110                 print(f"'{item}' popped from stack.")
111             except IndexError as e:
112                 print(e)
113         elif action == 'peek':
114             try:
115                 item = stack.peek()
116                 print(f"Top element is: '{item}'.")
117             except IndexError as e:
118                 print(e)
119         elif action == 'exit':
120             break
121         else:
122             print("Invalid action. Please try again.")
123
124
```

Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
1
True
Pop from an empty stack
Peek from an empty stack
PS C:\Users\sonu\Desktop\AI_Coding> & C:/Python314/python.exe "c:/Users/sonu/Desktop/AI_Coding/Stack Implementation.py"
Enter 'push' to add, 'pop' to remove, 'peek' to view top, 'exit' to quit: push
Enter the item to push: 2
'2' pushed to stack.
Enter 'push' to add, 'pop' to remove, 'peek' to view top, 'exit' to quit: push
Enter the item to push: 3
'3' pushed to stack.
Enter 'push' to add, 'pop' to remove, 'peek' to view top, 'exit' to quit: peek
Top element is: '3'.
Enter 'push' to add, 'pop' to remove, 'peek' to view top, 'exit' to quit: pop
'3' popped from stack.
Enter 'push' to add, 'pop' to remove, 'peek' to view top, 'exit' to quit: peek
Top element is: '2'.
Enter 'push' to add, 'pop' to remove, 'peek' to view top, 'exit' to quit: 
```

Task Description #2 – Queue Implementation Task: Use

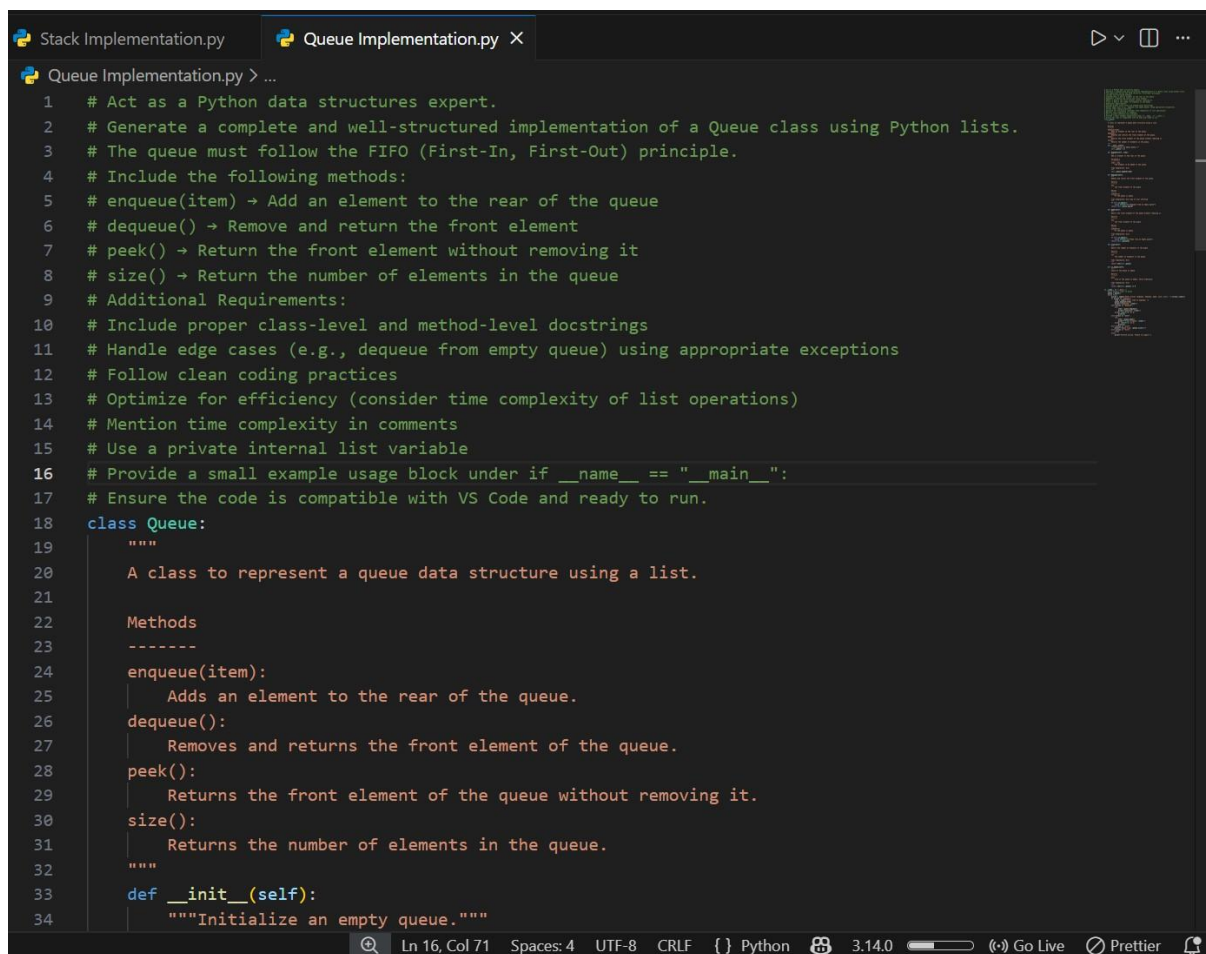
AI to implement a Queue using Python lists.

Sample Input Code: class

Queue: pass

Expected Output: • FIFO-based queue class with enqueue, dequeue, peek, and size methods.

Code:

A screenshot of a Visual Studio Code editor window. The top bar shows two tabs: 'Stack Implementation.py' and 'Queue Implementation.py X'. The active tab is 'Queue Implementation.py'. The editor displays a Python script for a Queue class. The code includes a docstring at the top with instructions for the AI, followed by the class definition and its methods. The status bar at the bottom indicates the cursor is at line 16, column 71, with settings for 4 spaces, UTF-8 encoding, CRLF line endings, and Python 3.14.0. The Queue class is implemented using a list and follows the FIFO principle.

```
1  # Act as a Python data structures expert.
2  # Generate a complete and well-structured implementation of a Queue class using Python lists.
3  # The queue must follow the FIFO (First-In, First-Out) principle.
4  # Include the following methods:
5  # enqueue(item) → Add an element to the rear of the queue
6  # dequeue() → Remove and return the front element
7  # peek() → Return the front element without removing it
8  # size() → Return the number of elements in the queue
9  # Additional Requirements:
10 # Include proper class-level and method-level docstrings
11 # Handle edge cases (e.g., dequeue from empty queue) using appropriate exceptions
12 # Follow clean coding practices
13 # Optimize for efficiency (consider time complexity of list operations)
14 # Mention time complexity in comments
15 # Use a private internal list variable
16 # Provide a small example usage block under if __name__ == "__main__":
17 # Ensure the code is compatible with VS Code and ready to run.
18 class Queue:
19     """
20     A class to represent a queue data structure using a list.
21
22     Methods
23     -----
24     enqueue(item):
25         Adds an element to the rear of the queue.
26     dequeue():
27         Removes and returns the front element of the queue.
28     peek():
29         Returns the front element of the queue without removing it.
30     size():
31         Returns the number of elements in the queue.
32     """
33     def __init__(self):
34         """Initialize an empty queue."""
```

```
Stack Implementation.py Queue Implementation.py X
Queue Implementation.py > ...
18 class Queue:
33     def __init__(self):
34         """Initialize an empty queue."""
35         self._queue = []
36
37     def enqueue(self, item):
38         """
39         Add an element to the rear of the queue.
40
41         Parameters
42         -----
43         item : any
44             The element to be added to the queue.
45
46         Time Complexity: O(1)
47         """
48         self._queue.append(item)
49
50     def dequeue(self):
51         """
52         Remove and return the front element of the queue.
53
54         Returns
55         -----
56         any
57             The front element of the queue.
58
59         Raises
60         -----
61         IndexError
62             If the queue is empty.
63
64         Time Complexity: O(n) due to list shifting
65         """
```

Ln 16, Col 71 Spaces: 4 UTF-8 CRLF Python 3.14.0 Go Live Prettier

```
Stack Implementation.py Queue Implementation.py X
Queue Implementation.py > ...
18 class Queue:
50     def dequeue(self):
66         if self.is_empty():
67             raise IndexError("Dequeue from an empty queue")
68         return self._queue.pop(0)
69
70     def peek(self):
71         """
72         Return the front element of the queue without removing it.
73
74         Returns
75         -----
76         any
77         | The front element of the queue.
78
79         Raises
80         -----
81         IndexError
82         | If the queue is empty.
83
84         Time Complexity: O(1)
85         """
86         if self.is_empty():
87             raise IndexError("Peek from an empty queue")
88         return self._queue[0]
89
90     def size(self):
91         """
92         Return the number of elements in the queue.
93
94         Returns
95         -----
96         int
97         | The number of elements in the queue.
```

Ln 16, Col 71 Spaces: 4 UTF-8 CRLF { } Python 3.14.0 Go Live Prettier

```
Stack Implementation.py Queue Implementation.py X
Queue Implementation.py > ...
18 class Queue:
90     def size(self):
91         """
92         Time Complexity: O(1)
93         """
94         return len(self._queue)
95
103     def is_empty(self):
104         """
105         Check if the queue is empty.
106
107         Returns
108         -----
109         bool
110         True if the queue is empty, False otherwise.
111
112         Time Complexity: O(1)
113         """
114         return len(self._queue) == 0
115
116 if __name__ == "__main__":
117     #give dynamic input to queue
118     queue = Queue()
119     while True:
120         action = input("Enter action (enqueue, dequeue, peek, size, exit): ").strip().lower()
121         if action == "enqueue":
122             item = input("Enter item to enqueue: ")
123             queue.enqueue(item)
124             print(f"Enqueued: {item}")
125         elif action == "dequeue":
126             try:
127                 item = queue.dequeue()
128                 print(f"Dequeued: {item}")
129             except IndexError as e:
130                 print(e)
```

Ln 16, Col 71 Spaces: 4 UTF-8 CRLF Python 3.14.0 Go Live Prettier

The screenshot displays a code editor with two tabs: "Stack Implementation.py" and "Queue Implementation.py". The "Queue Implementation.py" tab is active, showing a Python script for a queue. The code includes a `Queue` class with methods for enqueue, dequeue, peek, and size, along with a main loop that handles user input for these operations. The code is syntax-highlighted, and the editor includes a sidebar with a file explorer and a right-hand pane showing a diff or output view.

```
18 class Queue:
19     def __init__(self):
20         self.queue = []
21
22     def enqueue(self, item):
23         self.queue.append(item)
24
25     def dequeue(self):
26         if len(self.queue) == 0:
27             return None
28         return self.queue.pop(0)
29
30     def peek(self):
31         if len(self.queue) == 0:
32             return None
33         return self.queue[0]
34
35     def size(self):
36         return len(self.queue)
37
38     def is_empty(self):
39         return len(self.queue) == 0
40
41 if __name__ == "__main__":
42     #give dynamic input to queue
43     queue = Queue()
44     while True:
45         action = input("Enter action (enqueue, dequeue, peek, size, exit): ").strip().lower()
46         if action == "enqueue":
47             item = input("Enter item to enqueue: ")
48             queue.enqueue(item)
49             print(f"Enqueued: {item}")
50         elif action == "dequeue":
51             try:
52                 item = queue.dequeue()
53                 print(f"Dequeued: {item}")
54             except IndexError as e:
55                 print(e)
56         elif action == "peek":
57             try:
58                 item = queue.peek()
59                 print(f"Front element: {item}")
60             except IndexError as e:
61                 print(e)
62         elif action == "size":
63             print(f"Queue size: {queue.size()}")
64         elif action == "exit":
65             break
66         else:
67             print("Invalid action. Please try again.")
```

Ln 16, Col 71 Spaces: 4 UTF-8 CRLF {} Python 3.14.0 Go Live Prettier

Output:

```
14 # Mention time complexity in comments

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python + - [] [] ... | {} X

PS C:\Users\sonu\Desktop\AI_Coding> & C:/Python314/python.exe "c:/Users/sonu/Desktop/AI_Coding/Queue Implementation.py"
Enter action (enqueue, dequeue, peek, size, exit): enqueue
Enter item to enqueue: 3
Enqueued: 3
Enter action (enqueue, dequeue, peek, size, exit): size
Queue size: 1
Enter action (enqueue, dequeue, peek, size, exit): enqueue
Enter item to enqueue: 4
Enqueued: 4
Enter action (enqueue, dequeue, peek, size, exit): peek
Front element: 3
Enter action (enqueue, dequeue, peek, size, exit): exit
PS C:\Users\sonu\Desktop\AI_Coding>
```


Task Description #3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display methods.

Sample Input Code:

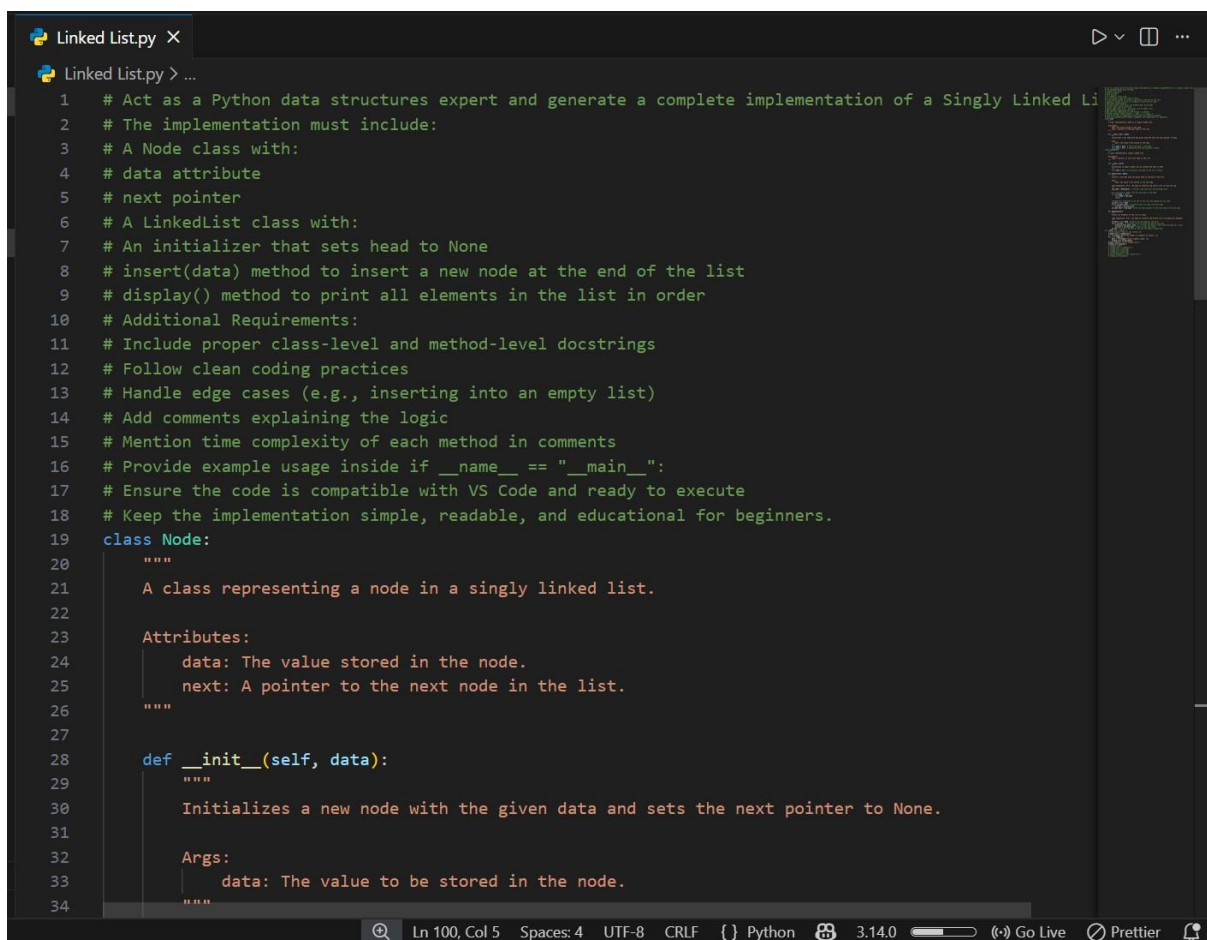
```
class Node: pass
```

```
LinkedList: pass
```

Expected Output:

- A working linked list implementation with clear method documentation.

Code:



```
1 # Act as a Python data structures expert and generate a complete implementation of a Singly Linked List
2 # The implementation must include:
3 # A Node class with:
4 # data attribute
5 # next pointer
6 # A LinkedList class with:
7 # An initializer that sets head to None
8 # insert(data) method to insert a new node at the end of the list
9 # display() method to print all elements in the list in order
10 # Additional Requirements:
11 # Include proper class-level and method-level docstrings
12 # Follow clean coding practices
13 # Handle edge cases (e.g., inserting into an empty list)
14 # Add comments explaining the logic
15 # Mention time complexity of each method in comments
16 # Provide example usage inside if __name__ == "__main__":
17 # Ensure the code is compatible with VS Code and ready to execute
18 # Keep the implementation simple, readable, and educational for beginners.
19 class Node:
20     """
21     A class representing a node in a singly linked list.
22
23     Attributes:
24         data: The value stored in the node.
25         next: A pointer to the next node in the list.
26     """
27
28     def __init__(self, data):
29         """
30         Initializes a new node with the given data and sets the next pointer to None.
31
32         Args:
33             data: The value to be stored in the node.
34         """
```

```
Linked List.py X
Linked List.py > ...
19 class Node:
20     def __init__(self, data):
35         self.data = data # Store the data in the node
36         self.next = None # Initialize the next pointer to None
37 class LinkedList:
38     """
39     A class representing a singly linked list.
40
41     Attributes:
42         head: A pointer to the first node in the list.
43     """
44
45     def __init__(self):
46         """
47         Initializes an empty linked list by setting the head to None.
48         """
49         self.head = None # Initialize the head of the list to None
50
51     def insert(self, data):
52         """
53         Inserts a new node with the given data at the end of the list.
54
55         Args:
56             data: The value to be stored in the new node.
57
58         Time Complexity: O(n) - We need to traverse the entire list to find the end.
59         """
60         new_node = Node(data) # Create a new node with the provided data
61
62         # If the list is empty, set the new node as the head
63         if self.head is None:
64             self.head = new_node
65         return
66
```

Ln 100, Col 5 Spaces: 4 UTF-8 CRLF { } Python 3.14.0 Go Live Prettier

```
Linked List.py X
Linked List.py > ...
37 class LinkedList:
51     def insert(self, data):
66
67         # Otherwise, traverse to the end of the list and append the new node
68         current = self.head
69         while current.next: # Traverse until we reach the last node
70             current = current.next
71         current.next = new_node # Set the next pointer of the last node to the new node
72
73     def display(self):
74         """
75         Prints all elements in the list in order.
76
77         Time Complexity: O(n) - We need to traverse the entire list to display all elements.
78         """
79         current = self.head # Start from the head of the list
80         while current: # Traverse until we reach the end of the list
81             print(current.data, end=' ') # Print the data of each node followed by a space
82             current = current.next # Move to the next node
83         print() # Print a newline at the end for better formatting
84 if __name__ == "__main__":
85     # give dynamic input to the linked list
86     linked_list = LinkedList()
87     n = int(input("Enter the number of elements to insert: "))
88     for i in range(n):
89         data = int(input(f"Enter element {i+1}: "))
90         linked_list.insert(data)
91     print("Elements in the linked list:")
92     linked_list.display()
93     # Example usage:
94     # linked_list = LinkedList()
95     # linked_list.insert(10)
96     # linked_list.insert(20)
97     # linked_list.insert(30)
```

Ln 100, Col 5 Spaces: 4 UTF-8 CRLF {} Python 3.14.0 Go Live Prettier

```
Linked List.py X
Linked List.py > ...
37 class LinkedList:
73     def display(self):
77         Time Complexity: O(n) - We need to traverse the entire list to display all elements.
78         """
79         current = self.head # Start from the head of the list
80         while current: # Traverse until we reach the end of the list
81             print(current.data, end=' ') # Print the data of each node followed by a space
82             current = current.next # Move to the next node
83         print() # Print a newline at the end for better formatting
84 if __name__ == "__main__":
85     # give dynamic input to the linked list
86     linked_list = LinkedList()
87     n = int(input("Enter the number of elements to insert: "))
88     for i in range(n):
89         data = int(input(f"Enter element {i+1}: "))
90         linked_list.insert(data)
91     print("Elements in the linked list:")
92     linked_list.display()
93     # Example usage:
94     # linked_list = LinkedList()
95     # linked_list.insert(10)
96     # linked_list.insert(20)
97     # linked_list.insert(30)
98     # print("Elements in the linked list:")
99     # linked_list.display()
100
```

Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\sonu\Desktop\AI_Coding> & C:/Python314/python.exe "c:/Users/sonu/Desktop/AI_Coding/Linked List
.py"
Enter the number of elements to insert: 2
Enter element 1: 3
Enter element 2: 4
Elements in the linked list:
3 4
PS C:\Users\sonu\Desktop\AI_Coding>
```

Task Description #4 – Binary Search Tree (BST)

Task: Use AI to create a BST with insert and in-order traversal methods.

Sample Input Code:

```
class BST: pass
```

Expected Output: • BST implementation with recursive insert and traversal methods.

Code:

```
Binary Search Tree (BST).py X
Binary Search Tree (BST).py > ...
1  # Act as a Python data structures expert.
2  # Generate a complete implementation of a Binary Search Tree (BST) in Python.
3  # The implementation must:
4  # Define a BST class
5  # Use a separate internal Node structure (with data, left, and right attributes)
6  # Implement a recursive insert(value) method that follows BST properties:
7  # Left subtree contains smaller values
8  # Right subtree contains larger values
9  # Implement a recursive inorder_traversal() method that prints elements in sorted order
10 # Additional Requirements:
11 # Use recursion for both insertion and traversal
12 # Include proper class-level and method-level docstrings
13 # Handle edge cases (e.g., inserting into an empty tree)
14 # Add comments explaining the recursive logic
15 # Mention time complexity in comments (Best, Average, Worst case)
16 # Follow clean coding practices
17 # Include example usage under if __name__ == "__main__":
18 # Ensure the code runs correctly in VS Code
19 # Keep the implementation beginner-friendly but conceptually accurate.
20 class Node:
21     """
22     A Node in a Binary Search Tree (BST).
23
24     Attributes:
25         data (int): The value stored in the node.
26         left (Node): Reference to the left child node.
27         right (Node): Reference to the right child node.
28     """
29
30     def __init__(self, data):
31         """
32         Initializes a new Node with the given data.
33
34         Args:
```

```
Binary Search Tree (BST).py X
Binary Search Tree (BST).py > ...
20 class Node:
30     def __init__(self, data):
34         Args:
35             data (int): The value to be stored in the node.
36         """
37         self.data = data
38         self.left = None
39         self.right = None
40
41 class BinarySearchTree:
42     """
43     A Binary Search Tree (BST) implementation.
44
45     Methods:
46         insert(value): Inserts a value into the BST.
47         inorder_traversal(): Prints the elements of the BST in sorted order.
48     """
49
50     def __init__(self):
51         """
52         Initializes an empty Binary Search Tree.
53         """
54         self.root = None
55
56     def insert(self, value):
57         """
58         Inserts a value into the BST while maintaining the BST properties.
59
60         Args:
61             value (int): The value to be inserted into the BST.
62
63         Time Complexity:
64             Best Case:  $O(\log n)$  - when the tree is balanced
65             Average Case:  $O(\log n)$  - when the tree is reasonably balanced
66             Worst Case:  $O(n)$  - when the tree is skewed
```



```
Binary Search Tree (BST).py ×
Binary Search Tree (BST).py > ...
41 class BinarySearchTree:
56     def insert(self, value):
57         """
58         Worst case: O(n) - when the tree becomes a linked list (unbalanced)
59         """
60         if self.root is None:
61             # If the tree is empty, set the root to a new node with the given value
62             self.root = Node(value)
63         else:
64             # Otherwise, call the recursive helper function to insert the value
65             self._insert_recursive(self.root, value)
66
67     def _insert_recursive(self, current_node, value):
68         """
69         Helper method to recursively insert a value into the BST.
70
71         Args:
72             current_node (Node): The current node being compared.
73             value (int): The value to be inserted.
74         """
75         if value < current_node.data:
76             # If the value is smaller than the current node's data, go to the left subtree
77             if current_node.left is None:
78                 current_node.left = Node(value)
79             else:
80                 self._insert_recursive(current_node.left, value)
81         else:
82             # If the value is greater than or equal to the current node's data, go to the right subtree
83             if current_node.right is None:
84                 current_node.right = Node(value)
85             else:
86                 self._insert_recursive(current_node.right, value)
87
88     def inorder_traversal(self):
89         """
90         Prints the elements of the BST in sorted order using an inorder traversal
91         """
92
93
94
95
96
97
98
```



```
Binary Search Tree (BST).py X
Binary Search Tree (BST).py > ...
41 class BinarySearchTree:
96     def inorder_traversal(self):
97         """
98         Prints the elements of the BST in sorted order using an inorder traversal.
99
100         Time Complexity:
101             Best Case: O(n) - when all nodes are visited
102             Average Case: O(n) - when all nodes are visited
103             Worst Case: O(n) - when all nodes are visited
104         """
105         self._inorder_recursive(self.root)
106
107     def _inorder_recursive(self, current_node):
108         """
109         Helper method to recursively perform an inorder traversal of the BST
110         and print the values.
111         Args:
112             current_node (Node): The current node being visited.
113         """
114         if current_node is not None:
115             # Traverse the left subtree first
116             self._inorder_recursive(current_node.left)
117             # Print the current node's data
118             print(current_node.data, end=' ')
119             # Traverse the right subtree next
120             self._inorder_recursive(current_node.right)
121
122 if __name__ == "__main__":
123     #give me dynamic input for the values to be inserted into the BST
124     bst = BinarySearchTree()
125     values = input("Enter values to insert into the BST (separated by spaces): ")
126     for value in values.split():
127         bst.insert(int(value))
128     print("Inorder Traversal of the BST:")
129     bst.inorder_traversal()
```

```
Binary Search Tree (BST).py X
Binary Search Tree (BST).py > ...
41 class BinarySearchTree:
107     def _inorder_recursive(self, current_node):
109         Helper method to recursively perform an inorder traversal of the BST
110         and print the values.
111         Args:
112             current_node (Node): The current node being visited.
113         """
114         if current_node is not None:
115             # Traverse the left subtree first
116             self._inorder_recursive(current_node.left)
117             # Print the current node's data
118             print(current_node.data, end=' ')
119             # Traverse the right subtree next
120             self._inorder_recursive(current_node.right)
121
122 if __name__ == "__main__":
123     #give me dynamic input for the values to be inserted into the BST
124     bst = BinarySearchTree()
125     values = input("Enter values to insert into the BST (separated by spaces): ")
126     for value in values.split():
127         bst.insert(int(value))
128     print("Inorder Traversal of the BST:")
129     bst.inorder_traversal()
```

Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python + v [Icons] | [Dark Theme] X
```

```
PS C:\Users\sonu\Desktop\AI_Coding> & C:/Python314/python.exe "c:/Users/sonu/Desktop/AI_Coding/Binary Search Tree (BST).py"
```

```
"
```

```
Enter values to insert into the BST (separated by spaces): 10 8 6 5 3 4
```

```
Inorder Traversal of the BST:
```

```
3 4 5 6 8 10
```

```
PS C:\Users\sonu\Desktop\AI_Coding>
```

```
[Search Icon] Ln 18, Col 44 Spaces: 4 UTF-8 CRLF {} Python [Icon] 3.14.0 [Progress Bar] Go Live Prettier [Icon]
```

Task Description #5 – Hash Table

Task: Use AI to implement a hash table with basic insert, search, and delete methods.

Sample Input Code: class

HashTable: pass

Expected Output: • Collision handling using chaining, with well-commented methods.

Code:

```
Hash Table.py X
Hash Table.py > ...
1 # Act as a Python data structures expert.
2 # Generate a complete implementation of a HashTable class in Python.
3 # The hash table must:
4 # Use a fixed-size list as the underlying storage
5 # Implement a custom hash function
6 # Handle collisions using separate chaining (each index should store a list of key-value pairs)
7 # Required methods:
8 # insert(key, value) → Insert or update a key-value pair
9 # search(key) → Return the value associated with the key
10 # delete(key) → Remove the key-value pair
11 # Additional Requirements:
12 # Include proper class-level and method-level docstrings
13 # Add detailed comments explaining the hashing and collision handling logic
14 # Handle edge cases (e.g., searching or deleting a non-existent key)
15 # Follow clean coding practices
16 # Mention time complexity for each operation (Average and Worst case)
17 # Use clear variable naming
18 # Include example usage inside if __name__ == "__main__":
19 # Ensure the implementation runs correctly in VS Code
20 # Keep the code beginner-friendly but technically accurate.
21 class HashTable:
22     """
23     A HashTable implementation using separate chaining for collision resolution.
24
25     Attributes:
26         size (int): The size of the underlying storage list.
27         table (list): The list that stores key-value pairs as lists (for separate chaining).
28     """
29     def __init__(self, size=10):
30         """
31         Initializes the HashTable with a specified size.
32
33         Args:
34             size (int): The size of the hash table. Default is 10.
```

```
Hash Table.py X
Hash Table.py > ...
21 class HashTable:
29     def __init__(self, size=10):
35         """
36         self.size = size
37         self.table = [[] for _ in range(size)] # Create a list of empty lists for separate chaining
38
39     def _hash(self, key):
40         """
41         Custom hash function to compute the index for a given key.
42
43         Args:
44             key: The key to be hashed.
45
46         Returns:
47             int: The computed index for the key.
48         """
49         return hash(key) % self.size # Use built-in hash and mod by size to get index
50
51     def insert(self, key, value):
52         """
53         Inserts or updates a key-value pair in the hash table.
54
55         Args:
56             key: The key to be inserted or updated.
57             value: The value associated with the key.
58
59         Time Complexity:
60             Average Case: O(1)
61             Worst Case: O(n) (when all keys hash to the same index)
62         """
63         index = self._hash(key) # Get the index for the key
64         bucket = self.table[index] # Get the bucket (list) at that index
65
66         # Check if the key already exists in the bucket
```

Ln 20, Col 60 Spaces: 4 UTF-8 CRLF {} Python 3.14.0 Go Live Prettier

```
Hash Table.py X
Hash Table.py > ...
21 class HashTable:
51     def insert(self, key, value):
65
66         # Check if the key already exists in the bucket
67         for i, (k, v) in enumerate(bucket):
68             if k == key: # If key exists, update the value
69                 bucket[i] = (key, value)
70             return
71
72         # If key does not exist, add a new key-value pair to the bucket
73         bucket.append((key, value))
74
75     def search(self, key):
76         """
77         Searches for a value associated with a given key.
78
79         Args:
80             key: The key to be searched.
81
82         Returns:
83             The value associated with the key if found, otherwise None.
84
85         Time Complexity:
86             Average Case: O(1)
87             Worst Case: O(n) (when all keys hash to the same index)
88         """
89         index = self._hash(key) # Get the index for the key
90         bucket = self.table[index] # Get the bucket (list) at that index
91
92         # Search for the key in the bucket
93         for k, v in bucket:
94             if k == key: # If key is found, return its value
95                 return v
96         return None # Return None if key is not found
```

Ln 20, Col 60 Spaces: 4 UTF-8 CRLF {} Python 3.14.0 Go Live Prettier

```
Hash Table.py X
Hash Table.py > ...
21 class HashTable:
75     def search(self, key):
95         return v
96     return None # Return None if key is not found
97
98     def delete(self, key):
99         """
100         Deletes a key-value pair from the hash table.
101
102         Args:
103             key: The key to be deleted.
104
105         Time Complexity:
106             Average Case: O(1)
107             Worst Case: O(n) (when all keys hash to the same index)
108         """
109         index = self._hash(key) # Get the index for the key
110         bucket = self.table[index] # Get the bucket (list) at that index
111
112         # Search for the key in the bucket and remove it if found
113         for i, (k, v) in enumerate(bucket):
114             if k == key: # If key is found, remove it from the bucket
115                 del bucket[i]
116             return
117         # If key is not found, do nothing (or you could raise an exception)
118
119 if __name__ == "__main__":
120     #give me dynamic input for the hash table
121     hash_table = HashTable()
122     while True:
123         action = input("Enter action (insert/search/delete/exit): ").strip().lower()
124         if action == "exit":
125             break
126         elif action in ["insert", "search", "delete"]:
```

Ln 20, Col 60 Spaces: 4 UTF-8 CRLF {} Python 3.14.0 Go Live Prettier


```
Hash Table.py X
Hash Table.py > ...
21 class HashTable:
98     def delete(self, key):
116         return
117         # If key is not found, do nothing (or you could raise an exception)
118
119 if __name__ == "__main__":
120     #give me dynamic input for the hash table
121     hash_table = HashTable()
122     while True:
123         action = input("Enter action (insert/search/delete/exit): ").strip().lower()
124         if action == "exit":
125             break
126         elif action in ["insert", "search", "delete"]:
127             key = input("Enter key: ")
128             if action == "insert":
129                 value = input("Enter value: ")
130                 hash_table.insert(key, value)
131                 print(f"Inserted ({key}, {value})")
132             elif action == "search":
133                 result = hash_table.search(key)
134                 if result is not None:
135                     print(f"Value for key '{key}': {result}")
136                 else:
137                     print(f"Key '{key}' not found.")
138             elif action == "delete":
139                 hash_table.delete(key)
140                 print(f"Deleted key '{key}' if it existed.")
141         else:
142             print("Invalid action. Please enter insert, search, delete, or exit.")
143
144
145
```

Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\sonu\Desktop\AI_Coding> & C:/Python314/python.exe "c:/Users/sonu/Desktop/AI_Coding/Hash Table.py"
Enter action (insert/search/delete/exit): insert
Enter key: 67
Enter value: 4
Inserted (67, 4)
Enter action (insert/search/delete/exit): insert
Enter key: 32
Enter value: 2
Inserted (32, 2)
Enter action (insert/search/delete/exit): search
Enter key: 5
Key '5' not found.
Enter action (insert/search/delete/exit): search
Enter key: 4
Key '4' not found.
Enter action (insert/search/delete/exit): search
Enter key: 67
Value for key '67': 4
Enter action (insert/search/delete/exit):
```

Task Description #6 – Graph Representation

Task: Use AI to implement a graph using an adjacency list.

Sample Input Code: class

Graph: pass

Expected Output: • Graph with methods to add vertices, add edges, and display connections.

Code:

```
Hash Table.py X
Hash Table.py > ...
1 # Act as a Python data structures expert.
2 # Generate a complete implementation of a HashTable class in Python.
3 # The hash table must:
4 # Use a fixed-size list as the underlying storage
5 # Implement a custom hash function
6 # Handle collisions using separate chaining (each index should store a list of key-value pairs)
7 # Required methods:
8 # insert(key, value) → Insert or update a key-value pair
9 # search(key) → Return the value associated with the key
10 # delete(key) → Remove the key-value pair
11 # Additional Requirements:
12 # Include proper class-level and method-level docstrings
13 # Add detailed comments explaining the hashing and collision handling logic
14 # Handle edge cases (e.g., searching or deleting a non-existent key)
15 # Follow clean coding practices
16 # Mention time complexity for each operation (Average and Worst case)
17 # Use clear variable naming
18 # Include example usage inside if __name__ == "__main__":
19 # Ensure the implementation runs correctly in VS Code
20 # Keep the code beginner-friendly but technically accurate.
21 class HashTable:
22     """
23     A HashTable implementation using separate chaining for collision resolution.
24
25     Attributes:
26         size (int): The size of the underlying storage list.
27         table (list): The list that stores key-value pairs as lists (for separate chaining).
28     """
29     def __init__(self, size=10):
30         """
31         Initializes the HashTable with a specified size.
32
33         Args:
34             size (int): The size of the hash table. Default is 10.
```

```
Hash Table.py X
Hash Table.py > ...
21 class HashTable:
29     def __init__(self, size=10):
34         size (int): The size of the hash table. Default is 10.
35         """
36         self.size = size
37         self.table = [[] for _ in range(size)] # Create a list of empty lists for separate chaining
38
39     def _hash(self, key):
40         """
41         Custom hash function to compute the index for a given key.
42
43         Args:
44             key: The key to be hashed.
45
46         Returns:
47             int: The computed index for the key.
48         """
49         return hash(key) % self.size # Use built-in hash and mod by size to get index
50
51     def insert(self, key, value):
52         """
53         Inserts or updates a key-value pair in the hash table.
54
55         Args:
56             key: The key to be inserted or updated.
57             value: The value associated with the key.
58
59         Time Complexity:
60             Average Case: O(1)
61             Worst Case: O(n) (when all keys hash to the same index)
62         """
63         index = self._hash(key) # Get the index for the key
64         bucket = self.table[index] # Get the bucket (list) at that index
65
```

Ln 20, Col 60 Spaces: 4 UTF-8 CRLF {} Python 3.14.0 Go Live Prettier

```
Hash Table.py X
Hash Table.py > ...
21 class HashTable:
51     def insert(self, key, value):
65
66         # Check if the key already exists in the bucket
67         for i, (k, v) in enumerate(bucket):
68             if k == key: # If key exists, update the value
69                 bucket[i] = (key, value)
70             return
71
72         # If key does not exist, add a new key-value pair to the bucket
73         bucket.append((key, value))
74
75     def search(self, key):
76         """
77         Searches for a value associated with a given key.
78
79         Args:
80             key: The key to be searched.
81
82         Returns:
83             The value associated with the key if found, otherwise None.
84
85         Time Complexity:
86             Average Case: O(1)
87             Worst Case: O(n) (when all keys hash to the same index)
88         """
89         index = self._hash(key) # Get the index for the key
90         bucket = self.table[index] # Get the bucket (list) at that index
91
92         # Search for the key in the bucket
93         for k, v in bucket:
94             if k == key: # If key is found, return its value
95                 return v
96         return None # Return None if key is not found
```

Ln 20, Col 60 Spaces: 4 UTF-8 CRLF {} Python 3.14.0 Go Live Prettier

```
Hash Table.py X
Hash Table.py > ...
21 class HashTable:
75     def search(self, key):
95         return v
96     return None # Return None if key is not found
97
98     def delete(self, key):
99         """
100         Deletes a key-value pair from the hash table.
101
102         Args:
103             key: The key to be deleted.
104
105         Time Complexity:
106             Average Case: O(1)
107             Worst Case: O(n) (when all keys hash to the same index)
108         """
109         index = self._hash(key) # Get the index for the key
110         bucket = self.table[index] # Get the bucket (list) at that index
111
112         # Search for the key in the bucket and remove it if found
113         for i, (k, v) in enumerate(bucket):
114             if k == key: # If key is found, remove it from the bucket
115                 del bucket[i]
116                 return
117         # If key is not found, do nothing (or you could raise an exception)
118
119 if __name__ == "__main__":
120     #give me dynamic input for the hash table
121     hash_table = HashTable()
122     while True:
123         action = input("Enter action (insert/search/delete/exit): ").strip().lower()
124         if action == "exit":
125             break
126         elif action in ["insert", "search", "delete"]:
```

Ln 20, Col 60 Spaces: 4 UTF-8 CRLF { } Python 3.14.0 Go Live Prettier

Task Description #7 – Priority Queue

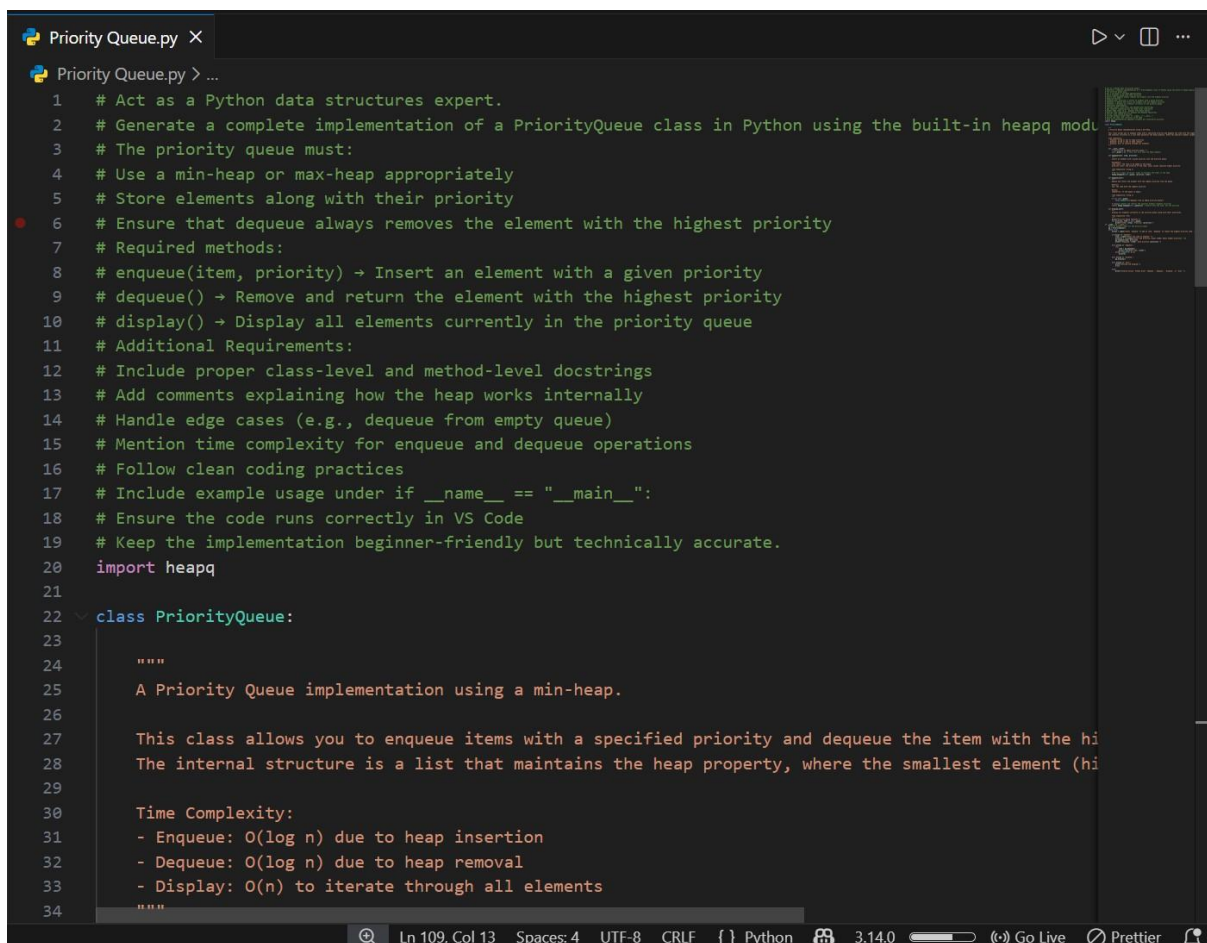
Task: Use AI to implement a priority queue using Python's heap module.

Sample Input Code:

```
class Priority Queue: pass
```

Expected Output: • Implementation with enqueue (priority), dequeue (highest priority) and display methods.

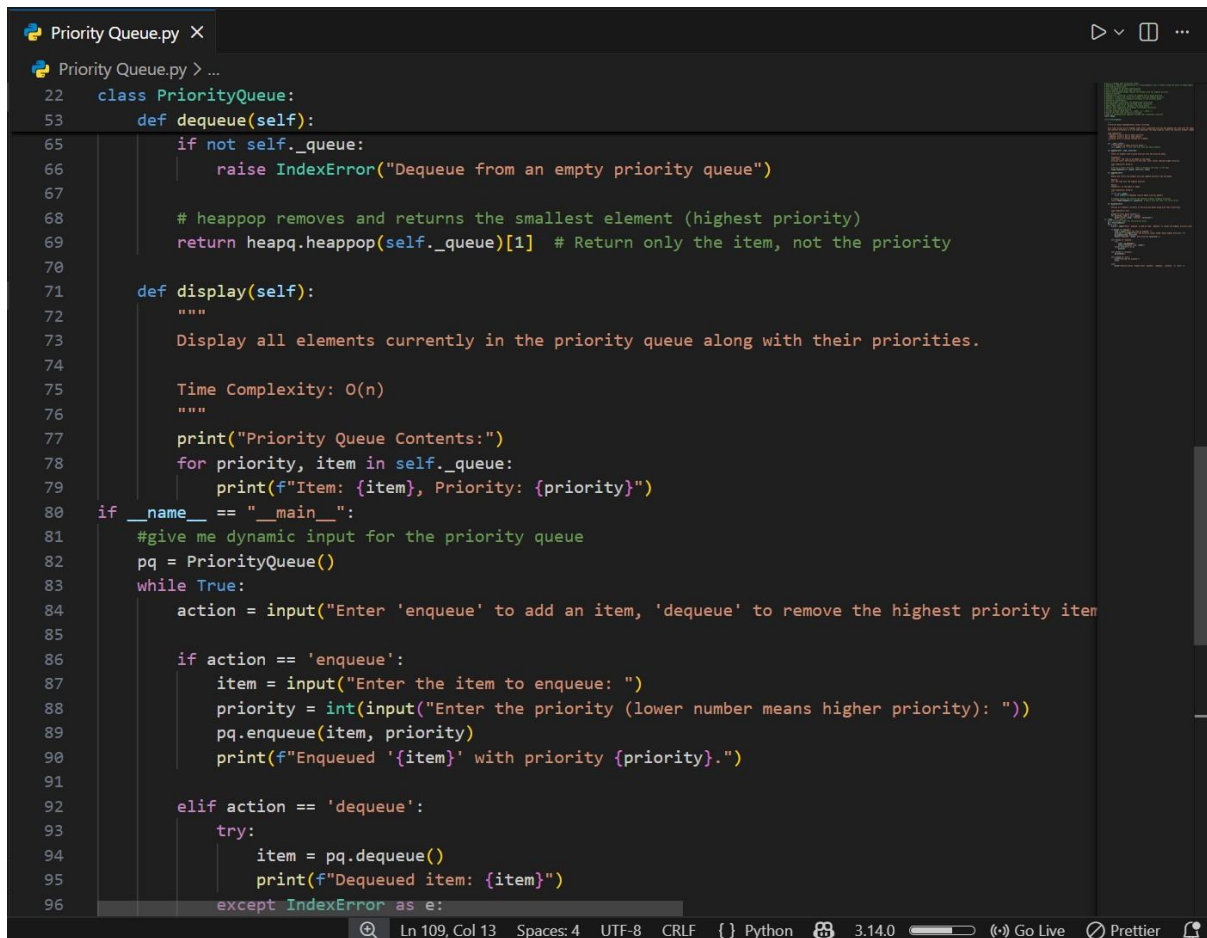
Code:



```
1  # Act as a Python data structures expert.
2  # Generate a complete implementation of a PriorityQueue class in Python using the built-in heapq module.
3  # The priority queue must:
4  # Use a min-heap or max-heap appropriately
5  # Store elements along with their priority
6  # Ensure that dequeue always removes the element with the highest priority
7  # Required methods:
8  # enqueue(item, priority) → Insert an element with a given priority
9  # dequeue() → Remove and return the element with the highest priority
10 # display() → Display all elements currently in the priority queue
11 # Additional Requirements:
12 # Include proper class-level and method-level docstrings
13 # Add comments explaining how the heap works internally
14 # Handle edge cases (e.g., dequeue from empty queue)
15 # Mention time complexity for enqueue and dequeue operations
16 # Follow clean coding practices
17 # Include example usage under if __name__ == "__main__":
18 # Ensure the code runs correctly in VS Code
19 # Keep the implementation beginner-friendly but technically accurate.
20 import heapq
21
22 class PriorityQueue:
23     """
24     A Priority Queue implementation using a min-heap.
25
26     This class allows you to enqueue items with a specified priority and dequeue the item with the highest priority.
27     The internal structure is a list that maintains the heap property, where the smallest element (highest priority) is at the root.
28
29     Time Complexity:
30     - Enqueue: O(log n) due to heap insertion
31     - Dequeue: O(log n) due to heap removal
32     - Display: O(n) to iterate through all elements
33     """
34
35     def __init__(self):
36         self._heap = []
37
38     def enqueue(self, item, priority):
39         """Enqueue an item with a given priority. The item is stored as a tuple (priority, item)."""
40         heapq.heappush(self._heap, (priority, item))
41
42     def dequeue(self):
43         """Dequeue the item with the highest priority (lowest value in the min-heap)."""
44         if not self._heap:
45             raise ValueError("Priority Queue is empty")
46         return heapq.heappop(self._heap)[1]
47
48     def display(self):
49         """Display all elements currently in the priority queue, sorted by priority (lowest to highest)."""
50         return self._heap
```

```
Priority Queue.py X
Priority Queue.py > ...
22 class PriorityQueue:
36     def __init__(self):
37         """Initialize an empty priority queue."""
38         self._queue = [] # This list will hold the heap elements
39
40     def enqueue(self, item, priority):
41         """
42         Insert an element with a given priority into the priority queue.
43
44         Parameters:
45         item (any): The item to be added to the queue.
46         priority (int): The priority of the item. Lower values indicate higher priority.
47
48         Time Complexity: O(log n)
49         """
50         # We use a tuple (priority, item) to maintain the order in the heap
51         heapq.heappush(self._queue, (priority, item))
52
53     def dequeue(self):
54         """
55         Remove and return the element with the highest priority from the queue.
56
57         Returns:
58         any: The item with the highest priority.
59
60         Raises:
61         IndexError: If the queue is empty.
62
63         Time Complexity: O(log n)
64         """
65         if not self._queue:
66             raise IndexError("Dequeue from an empty priority queue")
67
68         # heappop removes and returns the smallest element (highest priority)
```

```
Priority Queue.py X
Priority Queue.py > ...
22 class PriorityQueue:
53     def dequeue(self):
65         if not self._queue:
66             raise IndexError("Dequeue from an empty priority queue")
67
68         # heappop removes and returns the smallest element (highest priority)
69         return heapq.heappop(self._queue)[1] # Return only the item, not the priority
70
71     def display(self):
72         """
73         Display all elements currently in the priority queue along with their priorities.
74
75         Time Complexity: O(n)
76         """
77         print("Priority Queue Contents:")
78         for priority, item in self._queue:
79             print(f"Item: {item}, Priority: {priority}")
80 if __name__ == "__main__":
81     #give me dynamic input for the priority queue
82     pq = PriorityQueue()
83     while True:
84         action = input("Enter 'enqueue' to add an item, 'dequeue' to remove the highest priority item
85
86         if action == 'enqueue':
87             item = input("Enter the item to enqueue: ")
88             priority = int(input("Enter the priority (lower number means higher priority): "))
89             pq.enqueue(item, priority)
90             print(f"Enqueued '{item}' with priority {priority}.")
91
92         elif action == 'dequeue':
93             try:
94                 item = pq.dequeue()
95                 print(f"Dequeued item: {item}")
96             except IndexError as e:
```



Task Description #8 – Deque

Task: Use AI to implement a double-ended queue using collections. Deque.

Sample Input Code: class

Deque DS : pass

Expected Output: • Insert and remove from both ends with docstrings.

Code:

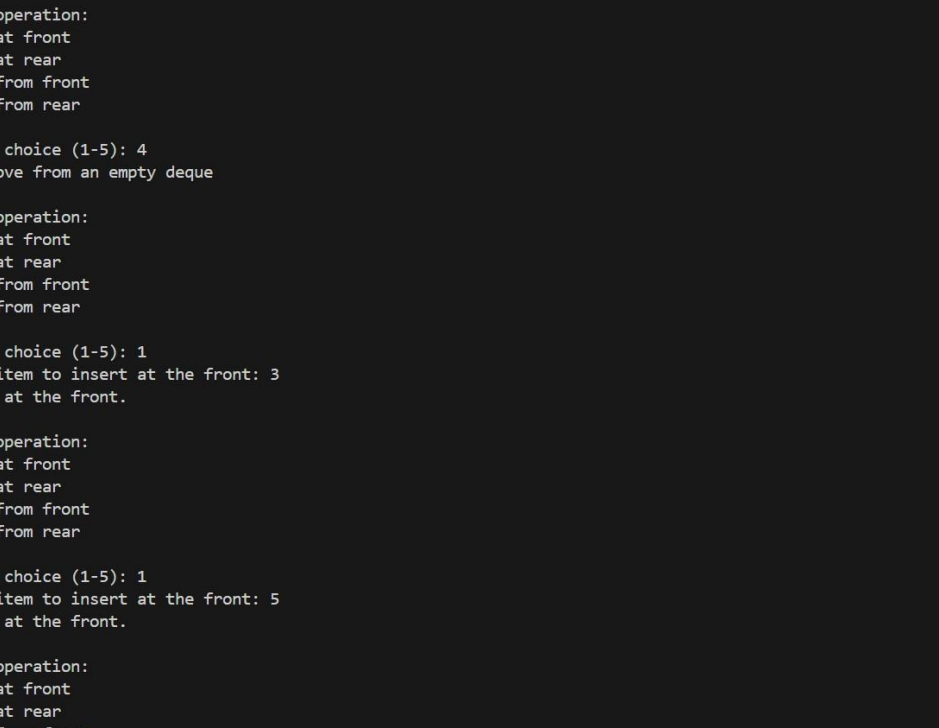
```
Deque.py X
Deque.py > ...
1 # Act as a Python data structures expert.
2 # Generate a complete implementation of a DequeDS class in Python using the built-in collections.deque
3 # The double-ended queue must:
4 # Allow insertion and deletion from both the front and rear ends
5 # Use collections.deque as the internal storage structure
6 # Required methods:
7 # insert_front(item) → Insert an element at the front
8 # insert_rear(item) → Insert an element at the rear
9 # remove_front() → Remove and return the front element
10 # remove_rear() → Remove and return the rear element
11 # Additional Requirements:
12 # Include proper class-level and method-level docstrings
13 # Add comments explaining how deque differs from a normal list
14 # Handle edge cases (e.g., removing from an empty deque)
15 # Mention time complexity for all operations
16 # Follow clean coding practices
17 # Include example usage under if __name__ == "__main__":
18 # Ensure the code runs correctly in VS Code
19 # Keep the implementation simple, readable, and suitable for beginners.
20 from collections import deque
21 class DequeDS:
22     """
23     A double-ended queue (deque) implementation using Python's built-in collections.deque.
24
25     A deque allows insertion and deletion of elements from both the front and rear ends.
26     The collections.deque is optimized for fast appends and pops from both ends, making it an ideal c
27
28     Time Complexity:
29     - insert_front(item): O(1)
30     - insert_rear(item): O(1)
31     - remove_front(): O(1)
32     - remove_rear(): O(1)
33     """
34
```

```
Deque.py X
Deque.py > ...
21 class DequeDS:
34
35     def __init__(self):
36         """Initialize an empty deque."""
37         self.deque = deque()
38
39     def insert_front(self, item):
40         """Insert an element at the front of the deque."""
41         self.deque.appendleft(item)
42
43     def insert_rear(self, item):
44         """Insert an element at the rear of the deque."""
45         self.deque.append(item)
46
47     def remove_front(self):
48         """Remove and return the front element of the deque. Raises IndexError if the deque is empty.
49         if not self.deque:
50             raise IndexError("Cannot remove from an empty deque")
51         return self.deque.popleft()
52
53     def remove_rear(self):
54         """Remove and return the rear element of the deque. Raises IndexError if the deque is empty."
55         if not self.deque:
56             raise IndexError("Cannot remove from an empty deque")
57         return self.deque.pop()
58 if __name__ == "__main__":
59     # give me dynamic input for the deque operations
60     deque_ds = DequeDS()
61     while True:
62         print("\nChoose an operation:")
63         print("1. Insert at front")
64         print("2. Insert at rear")
65         print("3. Remove from front")
66         print("4. Remove from rear")
```

Ln 18, Col 44 Spaces: 4 UTF-8 CRLF {} Python 3.14.0 Go Live Prettier


```
Deque.py X
Deque.py > ...
60 deque_ds = DequeDS()
61 while True:
62     print("\nChoose an operation:")
63     print("1. Insert at front")
64     print("2. Insert at rear")
65     print("3. Remove from front")
66     print("4. Remove from rear")
67     print("5. Exit")
68
69     choice = input("Enter your choice (1-5): ")
70
71     if choice == '1':
72         item = input("Enter the item to insert at the front: ")
73         deque_ds.insert_front(item)
74         print(f"Inserted {item} at the front.")
75     elif choice == '2':
76         item = input("Enter the item to insert at the rear: ")
77         deque_ds.insert_rear(item)
78         print(f"Inserted {item} at the rear.")
79     elif choice == '3':
80         try:
81             removed_item = deque_ds.remove_front()
82             print(f"Removed {removed_item} from the front.")
83         except IndexError as e:
84             print(e)
85     elif choice == '4':
86         try:
87             removed_item = deque_ds.remove_rear()
88             print(f"Removed {removed_item} from the rear.")
89         except IndexError as e:
90             print(e)
91     elif choice == '5':
92         print("Exiting...")
93         break
```

Ln 18, Col 44 Spaces: 4 UTF-8 CRLF {} Python 3.14.0 Go Live Prettier

Output:

```
PS C:\Users\sonu\Desktop\AI_Coding> & C:/Python314/python.exe c:/Users/sonu/Desktop/AI_Coding/Deque.py

Choose an operation:
1. Insert at front
2. Insert at rear
3. Remove from front
4. Remove from rear
5. Exit
Enter your choice (1-5): 4
Cannot remove from an empty deque

Choose an operation:
1. Insert at front
2. Insert at rear
3. Remove from front
4. Remove from rear
5. Exit
Enter your choice (1-5): 1
Enter the item to insert at the front: 3
Inserted 3 at the front.

Choose an operation:
1. Insert at front
2. Insert at rear
3. Remove from front
4. Remove from rear
5. Exit
Enter your choice (1-5): 1
Enter the item to insert at the front: 5
Inserted 5 at the front.

Choose an operation:
1. Insert at front
2. Insert at rear
3. Remove from front
4. Remove from rear
5. Exit
Enter your choice (1-5): 2
```


o Stack o Queue o Priority

Queue o Linked List o

Binary Search Tree (BST) o

Graph o Hash Table o

Deque

- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

Code:

```
Real-Time Application Challenge.py X
Real-Time Application Challenge.py > ...
9 # Graph
10 # Hash Table
11 # Deque
12 # System Features:
13 # Student Attendance Tracking - Daily log of students entering/exiting the campus.
14 # Event Registration System - Manage participants in events with quick search and removal.
15 # Library Book Borrowing - Keep track of available books and their due dates.
16 # Bus Scheduling System - Maintain bus routes and stop connections.
17 # Cafeteria Order Queue - Serve students in the order they arrive.
18 # Your Tasks:
19 # For each feature:
20 # Select the most appropriate data structure.
21 # Provide a clear justification in 2-3 sentences explaining why it fits.
22 # Mention relevant time complexity considerations.
23 # Create a well-formatted table mapping:
24 # Feature -> Selected Data Structure -> Justification
25 # Choose one feature and implement it as a complete working Python program with:
26 # Proper class structure
27 # Docstrings
28 # Comments explaining logic
29 # Edge case handling
30 # Time complexity notes
31 # Example usage inside if __name__ == "__main__":
32 # Ensure the implementation is beginner-friendly, efficient, and compatible with VS Code.
33 # Feature -> Selected Data Structure -> Justification
34 feature_mapping = [
35     ("Student Attendance Tracking", "Queue", "A queue is ideal for tracking students entering and exiting the campus."),
36     ("Event Registration System", "Hash Table", "A hash table allows for quick search, insertion, and removal of participants."),
37     ("Library Book Borrowing", "Binary Search Tree (BST)", "A BST can efficiently manage available books and their due dates."),
38     ("Bus Scheduling System", "Graph", "A graph is ideal for representing bus routes and stop connections."),
39     ("Cafeteria Order Queue", "Queue", "A queue is perfect for serving students in the order they arrive.")
40 ]
41 # Display the feature mapping in a well-formatted table
42 print(f'{"Feature":<30} {"Selected Data Structure":<25} {"Justification"}')
```

Ln 32, Col 90 Spaces: 4 UTF-8 CRLF {} Python 3.14.0 Go Live Prettier

```
Real-Time Application Challenge.py X
Real-Time Application Challenge.py > ...
40 ]
41 # Display the feature mapping in a well-formatted table
42 print(f"{'Feature':<30} {'Selected Data Structure':<25} {'Justification'}")
43 print("-" * 80)
44 for feature, data_structure, justification in feature_mapping:
45     print(f"{'feature':<30} {'data_structure':<25} {'justification'}")
46 # Implementing the Cafeteria Order Queue feature using a Queue data structure
47 class CafeteriaOrderQueue:
48     """
49     A class to represent a cafeteria order queue.
50     This class uses a list to implement the queue functionality.
51     """
52
53     def __init__(self):
54         """Initialize an empty queue."""
55         self.queue = []
56
57     def enqueue(self, order):
58         """
59         Add an order to the end of the queue.
60
61         :param order: The order to be added (string).
62         """
63         self.queue.append(order) # O(1) time complexity
64
65     def dequeue(self):
66         """
67         Remove and return the order at the front of the queue.
68
69         :return: The order at the front of the queue (string) or None if the queue is empty.
70         """
71         if not self.is_empty():
72             return self.queue.pop(0) # O(n) time complexity due to shifting elements
73         else:
```

Ln 32, Col 90 Spaces: 4 UTF-8 CRLF {} Python 3.14.0 Go Live Prettier


```
Real-Time Application Challenge.py X
Real-Time Application Challenge.py > ...
47 class CafeteriaOrderQueue:
65     def dequeue(self):
69         :return: The order at the front of the queue (string) or None if the queue is empty.
70         """
71         if not self.is_empty():
72             return self.queue.pop(0) # O(n) time complexity due to shifting elements
73         else:
74             print("The queue is empty. No orders to serve.")
75             return None
76
77     def is_empty(self):
78         """
79         Check if the queue is empty.
80
81         :return: True if the queue is empty, False otherwise.
82         """
83         return len(self.queue) == 0
84
85     def size(self):
86         """
87         Get the number of orders in the queue.
88
89         :return: The size of the queue (integer).
90         """
91         return len(self.queue)
92 if __name__ == "__main__":
93     #give me dynamic input for the cafeteria order queue
94     cafeteria_queue = CafeteriaOrderQueue()
95     while True:
96         action = input("Enter 'enqueue' to add an order, 'dequeue' to serve an order, or 'exit' to qu
97         if action == 'enqueue':
98             order = input("Enter the order to be added: ")
99             cafeteria_queue.enqueue(order)
100             print(f"Order '{order}' added to the queue.")
Ln 32, Col 90 Spaces: 4 UTF-8 CRLF {} Python 3.14.0 Go Live Prettier
```

```
Real-Time Application Challenge.py X
Real-Time Application Challenge.py > ...
47 class CafeteriaOrderQueue:
85     def size(self):
87         Get the number of orders in the queue.
88
89         :return: The size of the queue (integer).
90         """
91         return len(self.queue)
92 if __name__ == "__main__":
93     #give me dynamic input for the cafeteria order queue
94     cafeteria_queue = CafeteriaOrderQueue()
95     while True:
96         action = input("Enter 'enqueue' to add an order, 'dequeue' to serve an order, or 'exit' to qu
97         if action == 'enqueue':
98             order = input("Enter the order to be added: ")
99             cafeteria_queue.enqueue(order)
100             print(f"Order '{order}' added to the queue.")
101         elif action == 'dequeue':
102             served_order = cafeteria_queue.dequeue()
103             if served_order:
104                 print(f"Order '{served_order}' has been served.")
105         elif action == 'exit':
106             print("Exiting the cafeteria order queue system.")
107             break
108         else:
109             print("Invalid action. Please enter 'enqueue', 'dequeue', or 'exit'.")
110
```

Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python + v [Icons] | [Icons] X

PS C:\Users\sonu\Desktop\AI_Coding> & C:/Python314/python.exe "c:/Users/sonu/Desktop/AI_Coding/Real-Time Application Challenge.py"
Feature                               Selected Data Structure   Justification
-----
Student Attendance Tracking           Queue                    A queue is ideal for tracking students entering and exiting the campus as it follows the First-In-First-Out (FIFO) principle, allowing for efficient logging of attendance. Time complexity for enqueue and dequeue operations is O(1).
Event Registration System              Hash Table               A hash table allows for quick search, insertion, and deletion of participants in events, making it suitable for managing registrations. Time complexity for search, insert, and delete operations is O(1) on average.
Library Book Borrowing                 Binary Search Tree (BST) A BST can efficiently manage available books and their due dates by allowing for sorted data storage and quick retrieval. Time complexity for search, insert, and delete operations is O(log n) on average.
Bus Scheduling System                  Graph                    A graph is ideal for representing bus routes and stop connections as it can model complex relationships between stops and routes. Time complexity for traversal operations is O(V + E), where V is the number of vertices (stops) and E is the number of edges (routes).
Cafeteria Order Queue                  Queue                    A queue is perfect for serving students in the order they arrive at the cafeteria, ensuring fairness. Time complexity for enqueue and dequeue operations is O(1).
Enter 'enqueue' to add an order, 'dequeue' to serve an order, or 'exit' to quit: enqueue
Enter the order to be added: 5 7 9 10
Order '5 7 9 10' added to the queue.
Enter 'enqueue' to add an order, 'dequeue' to serve an order, or 'exit' to quit: dequeue
Order '5 7 9 10' has been served.
Enter 'enqueue' to add an order, 'dequeue' to serve an order, or 'exit' to quit: exit
Exiting the cafeteria order queue system.
PS C:\Users\sonu\Desktop\AI_Coding>
```

Task Description #10: Smart E-Commerce Platform – Data Structure Challenge

An e-commerce company wants to build a Smart Online Shopping System with:

1. Shopping Cart Management – Add and remove products dynamically.
2. Order Processing System – Orders processed in the order they are placed.
3. Top-Selling Products Tracker – Products ranked by sales count.
4. Product Search Engine – Fast lookup of products using product ID.
5. Delivery Route Planning – Connect warehouses and delivery locations.

Student Task:

- For each feature, select the most appropriate data structure from the list below:

o Stack o Queue o Priority

Queue o Linked List o

Binary Search Tree (BST) o

Graph o Hash Table o

Deque

- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

Code:

```
Smart E-Commerce Platform .py X
Smart E-Commerce Platform .py > ...
1 # Act as a data structures expert and system architect.
2 # Analyze the following Smart E-Commerce Platform requirements and select the most appropriate data s
3 # Available Data Structures:
4 # Stack
5 # Queue
6 # Priority Queue
7 # Linked List
8 # Binary Search Tree (BST)
9 # Graph
10 # Hash Table
11 # Deque
12 # System Features:
13 # Shopping Cart Management - Add and remove products dynamically.
14 # Order Processing System - Orders processed in the order they are placed.
15 # Top-Selling Products Tracker - Products ranked by sales count.
16 # Product Search Engine - Fast lookup of products using product ID.
17 # Delivery Route Planning - Connect warehouses and delivery locations.
18 # Required Tasks:
19 # For each feature:
20 # Select the most appropriate data structure.
21 # Provide a clear 2-3 sentence justification.
22 # Mention relevant time complexity considerations.
23 # Present the results in a well-formatted table:
24 # Feature → Selected Data Structure → Justification
25 # Choose one feature and implement it as a complete Python program with:
26 # Proper class-based structure
27 # Docstrings for class and methods
28 # Inline comments explaining logic
29 # Edge case handling
30 # Time complexity notes
31 # Example usage under if __name__ == "__main__":
32 # Code compatible with VS Code
33 # Ensure the solution is technically accurate, beginner-friendly, and follows clean coding practices.
34 # Feature → Selected Data Structure → Justification
```

```
Smart E-Commerce Platform .py X
Smart E-Commerce Platform.py > ...
33 # Ensure the solution is technically accurate, beginner-friendly, and follows clean coding practices.
34 # Feature → Selected Data Structure → Justification
35 features_data_structures = [
36     ("Shopping Cart Management", "Linked List", "A linked list allows for dynamic addition and removal of items in a shopping cart."),
37     ("Order Processing System", "Queue", "A queue is ideal for processing orders in the order they are received."),
38     ("Top-Selling Products Tracker", "Priority Queue", "A priority queue can efficiently keep track of top-selling products based on sales volume or revenue."),
39     ("Product Search Engine", "Hash Table", "A hash table allows for fast lookup of products using product IDs or keywords as keys."),
40     ("Delivery Route Planning", "Graph", "A graph is suitable for modeling warehouses and delivery locations, enabling efficient route planning algorithms like Dijkstra's or A* search.")
41 ]
42 # Display the results in a well-formatted table
43 print(f'{"Feature":<30} {"Selected Data Structure":<25} {"Justification"}')
44 for feature, data_structure, justification in features_data_structures:
45     print(f'{"feature":<30} {"data_structure":<25} {"justification"}')
46 # Implementing the Shopping Cart Management feature using a Linked List
47 class Node:
48     """A node in the linked list representing a product in the shopping cart."""
49     def __init__(self, product_id, quantity):
50         self.product_id = product_id
51         self.quantity = quantity
52         self.next = None
53 class ShoppingCart:
54     """A shopping cart implemented as a linked list."""
55     def __init__(self):
56         self.head = None
57     def add_product(self, product_id, quantity):
58         """Add a product to the shopping cart."""
59         new_node = Node(product_id, quantity)
60         new_node.next = self.head
61         self.head = new_node
62     def remove_product(self, product_id):
63         """Remove a product from the shopping cart."""
64         current = self.head
65         previous = None
66         while current is not None:
```



```
Smart E-Commerce Platform .py X
Smart E-Commerce Platform .py > ...
53 class ShoppingCart:
54     self.head = new_node
55
56     def remove_product(self, product_id):
57         """Remove a product from the shopping cart."""
58         current = self.head
59         previous = None
60         while current is not None:
61             if current.product_id == product_id:
62                 if previous is None: # Removing the head node
63                     self.head = current.next
64                 else:
65                     previous.next = current.next
66                 return True # Product removed successfully
67             previous = current
68             current = current.next
69         return False # Product not found in the cart
70
71     def display_cart(self):
72         """Display the contents of the shopping cart."""
73         products = []
74         current = self.head
75         while current is not None:
76             products.append(f"Product ID: {current.product_id}, Quantity: {current.quantity}")
77             current = current.next
78         return products
79
80 if __name__ == "__main__":
81     #give me dynamic input for the shopping cart management
82     cart = ShoppingCart()
83     while True:
84         action = input("Enter 'add' to add a product, 'remove' to remove a product, 'display' to show
85         if action == 'add':
86             product_id = input("Enter product ID: ")
87             quantity = int(input("Enter quantity: "))
88             cart.add_product(product_id, quantity)
89             print(f"Added Product ID: {product_id} with Quantity: {quantity}")
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
2520
2521
2522
2523
2524
2525
2526
2527
2528
2529
2530
2531
2532
2533
2534
2535
2536
2537
2538
2539
2540
2541
2542
2543
2544
2545
2546
2547
2548
2549
2550
2551
2552
2553
2554
2555
2556
2557
2558
2559
2560
2561
2562
2563
2564
2565
2566
2567
2568
2569
2570
2571
2572
2573
2574
2575
2576
2577
2578
2579
2580
2581
2582
2583
2584
2585
2586
2587
2588
2589
2590
2591
2592
2593
2594
2595
2596
2597
2598
2599
2600
2601
2602
2603
2604
2605
2606
2607
2608
2609
2610
2611
2612
2613
2614
2615
2616
2617
2618
2619
2620
2621
2622
2623
2624
2625
2626
2627
2628
2629
2630
2631
2632
2633
2634
2635
2636
2637
2638
2639
2640
2641
2642
2643
2644
2645
2646
2647
2648
2649
2650
2651
2652
2653
2654
2655
2656
2657
2658
2659
2660
2661
2662
266
```

Python + ... |

Feature	Selected Data Structure	Justification
Shopping Cart Management without needing to shift elements, making it efficient for managing a shopping cart. Time complexity for adding/removing products is $O(1)$.	Linked List	A linked list allows for dynamic addition and removal of products

Top-Selling Products Tracker **Priority Queue** A priority queue can efficiently keep track of top-selling products by maintaining a sorted order based on sales count. Time complexity for insertion and retrieval is $O(\log n)$.

Delivery Route Planning	Graph	A graph is suitable for modeling warehouses and delivery locations, allowing for efficient route planning using algorithms like Dijkstra's or A*. Time complexity depends on the algorithm
-------------------------	-------	--

Enter 'add' to add a product, 'remove' to remove a product, 'display' to show cart contents, or 'exit' to quit: add

Enter quantity: 300

```
Enter 'add' to add a product, 'remove' to remove a product, 'display' to show cart contents, or 'exit' to quit: display
```

```
Enter: add to add a product, remove to remove a product, display to show cart contents, or exit to quit. display
Shopping Cart Contents:
```

Product ID: 567903, Quantity: 300

```
Enter 'add' to add a product, 'remove' to remove a product, 'display' to show cart contents, or 'exit' to quit:
```